Name: Hifza Hussain

Roll no: SU92-BSAIM-F25-330

Section: BSAI-3A

Design and Implementation of a Stateful AI Controller

1 Introduction

A stateful AI controller is an intelligent system that maintains information about its previous actions or states to make better decisions. Unlike a simple reflex agent, which responds only to the current percept, a stateful controller uses both the current input and its previous state.

This question presents a Python-based temperature control system that remembers its previous operating mode, avoids sending duplicate commands, and handles boundary conditions.

2 System Requirements

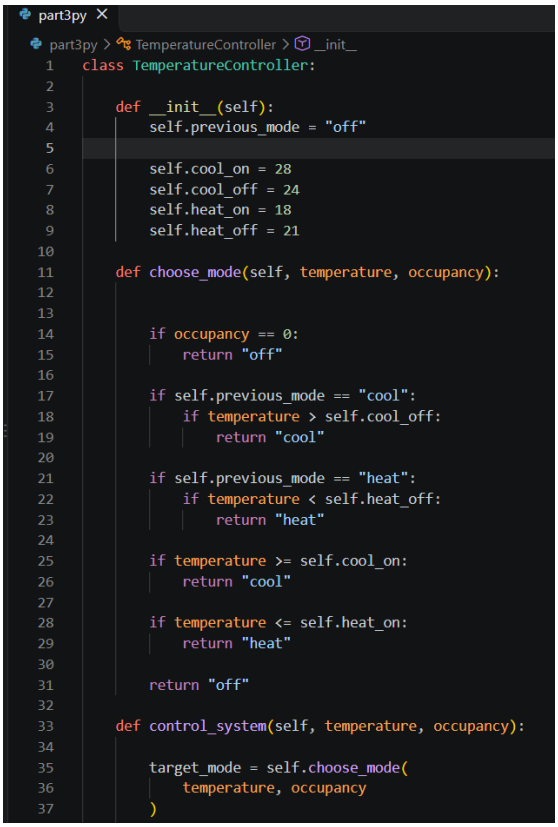
The controller follows these rules:

1. **Cooling Mode:** Activate cooling when the temperature reaches or exceeds 28°C. Keep cooling active until the temperature reaches 24°C or below.
2. **Heating Mode:** Activate heating when the temperature reaches or falls below 18°C. Keep heating active until the temperature reaches 21°C or above.
3. **Occupancy Priority:** If no people are present, set the target mode to off, regardless of temperature.
4. **State Memory:** Remember the previous operating mode.
5. **Command Optimization:** Send a command only when the target mode differs from the previous mode.

Separate activation and deactivation thresholds create a technique called *hysteresis*, which helps prevent frequent switching when the temperature fluctuates near a threshold.

3 Python Implementation

C:\Users\DELL\Documents\ai lab\task 2\task2.py\part3py



```python
class TemperatureController:

    def __init__(self):
        self.previous_mode = "off"

        self.cool_on = 28
        self.cool_off = 24
        self.heat_on = 18
        self.heat_off = 21

    def choose_mode(self, temperature, occupancy):


        if occupancy == 0:
            return "off"

        if self.previous_mode == "cool":
            if temperature > self.cool_off:
                return "cool"

        if self.previous_mode == "heat":
            if temperature < self.heat_off:
                return "heat"

        if temperature >= self.cool_on:
            return "cool"

        if temperature <= self.heat_on:
            return "heat"

        return "off"

    def control_system(self, temperature, occupancy):

        target_mode = self.choose_mode(
            temperature, occupancy
        )
```



```python
        if target_mode != self.previous_mode:
            command = "SET_MODE " + target_mode
        else:
            command = "NO COMMAND"

        self.previous_mode = target_mode

        return target_mode, command

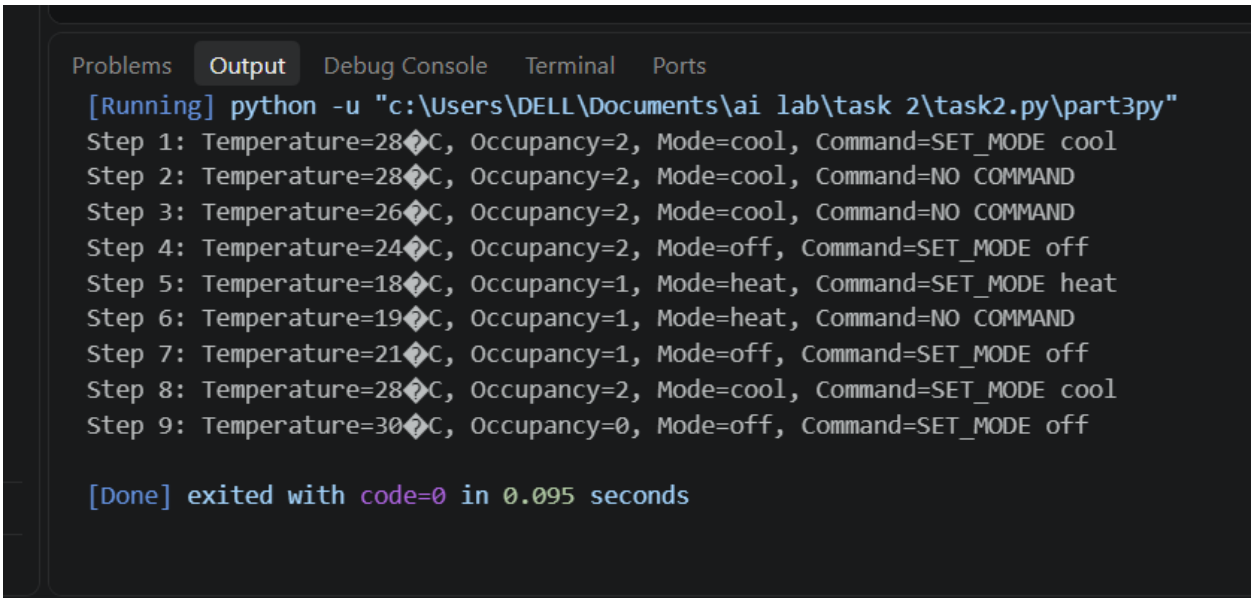

controller = TemperatureController()

test_cases = [
    (28, 2),
    (28, 2),
    (26, 2),
    (24, 2),
    (18, 1),
    (19, 1),
    (21, 1),
    (28, 2),
    (30, 0)
]

for i, (temperature, occupancy) in enumerate(test_cases, 1):

    target_mode, command = controller.control_system(
        temperature, occupancy
    )

    print(
        f"Step {i}: Temperature={temperature}°C, "
        f"Occupancy={occupancy}, "
        f"Mode={target_mode}, "
        f"Command={command}"
    )
```



```
Problems  Output  Debug Console  Terminal  Ports
[Running] python -u "c:\Users\DELL\Documents\ai lab\task 2\task2.py\part3py"
Step 1: Temperature=28°C, Occupancy=2, Mode=cool, Command=SET_MODE cool
Step 2: Temperature=28°C, Occupancy=2, Mode=cool, Command=NO COMMAND
Step 3: Temperature=26°C, Occupancy=2, Mode=cool, Command=NO COMMAND
Step 4: Temperature=24°C, Occupancy=2, Mode=off, Command=SET_MODE off
Step 5: Temperature=18°C, Occupancy=1, Mode=heat, Command=SET_MODE heat
Step 6: Temperature=19°C, Occupancy=1, Mode=heat, Command=NO COMMAND
Step 7: Temperature=21°C, Occupancy=1, Mode=off, Command=SET_MODE off
Step 8: Temperature=28°C, Occupancy=2, Mode=cool, Command=SET_MODE cool
Step 9: Temperature=30°C, Occupancy=0, Mode=off, Command=SET_MODE off

[Done] exited with code=0 in 0.095 seconds
```

4 Test Cases and Expected Results

The following table shows the expected output for each test case.

Test Case	Temperature	Occupancy	Previous Mode	Expected Target Mode	Expected Command
Initial cooling	28°C	2	off	cool	SET_MODE cool
Repeated cool percept	28°C	2	cool	cool	NO COMMAND
Cooling continues	26°C	2	cool	cool	NO COMMAND
Cooling stops	24°C	1	cool	off	SET_MODE off
Heating starts	18°C		off	heat	SET_MODE heat
Heating continues	19°C	1	heat	heat	NO COMMAND
Heating stops	21°C	1	heat	off	SET_MODE off
Empty room	30°C	0	off	off	NO COMMAND

5. Handling Edge Cases

Case 1: Repeated Percepts

If the temperature remains at 28°C and the previous mode is already cool, the controller keeps the cooling mode active without sending another command.

Expected result: NO COMMAND

This prevents unnecessary repeated commands.

Case 2: Exact Cooling Boundary

If the temperature reaches exactly 24°C while the previous mode is cool, the controller switches to off.

Expected result: SET_MODE off

This confirms that the cooling deactivation threshold is handled correctly.

Case 3: Exact Heating Boundary

If the temperature reaches exactly 18°C while the previous mode is off, the controller activates heating.

Expected result: SET_MODE heat

This confirms that the heating activation threshold is inclusive.

Case 4: Empty Room

If the room is empty, the controller selects off, even if the temperature is 30°C.

Expected result: NO COMMAND when the previous mode is already off.

This demonstrates that occupancy takes priority over temperature in the specified control policy.

6 Alternative Design

An alternative approach is to use a simple reflex agent that selects a mode using only the current temperature and occupancy. However, such a controller does not remember its previous state and may switch modes unnecessarily near temperature boundaries.

The stateful approach is preferable because it remembers the previous mode, applies hysteresis, and sends commands only when a mode change is required. A more advanced system could also include minimum operating times, sensor validation, and manual controls.

7 Conclusion

The implemented Python controller demonstrates how an AI agent can use internal state to improve decision-making. It remembers the previous operating mode, applies separate activation and deactivation thresholds, prioritizes room occupancy, and avoids sending duplicate commands. The test cases cover repeated percepts, mode transitions, exact threshold values, and the empty-room condition. These features make the controller more consistent and efficient than a basic reflex-only approach.